# MẠNG PHÁT HIỆN ĐỐI TƯỢNG (OBJECT DETECTION)

Chủ đề Week 06 — Lý thuyết Chi tiết, Chỉ số Đánh giá, So sánh Kiến trúc & Code Example Python

## 1. Tổng quan Tác vụ Object Detection & Khái niệm Nền tảng

**Thị giác máy tính (Computer Vision)** chia làm 4 cấp độ tác vụ chính:
• **Image Classification:** Phân loại nhãn duy nhất cho toàn bộ bức ảnh (VD: 'Mèo').
• **Classification + Localization:** Xác định nhãn và vị trí khung Bounding Box duy nhất cho 1 vật thể.
• **Object Detection:** Định vị (Bounding Box) và Phân loại **nhiều vật thể đồng thời** trong ảnh.
• **Semantic / Instance Segmentation:** Phân loại ở cấp độ từng điểm ảnh (Pixel-level).

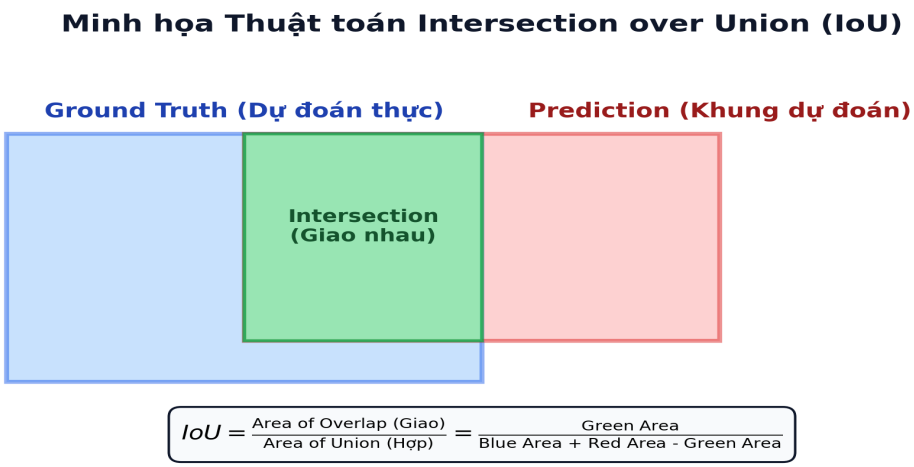
**Biểu diễn Bounding Box:** Mỗi Bounding Box được xác định bởi tọa độ 4 chiều: (x_min, y_min, x_max, y_max) hoặc (x_center, y_center, width, height).

> **Khái niệm Anchor Boxes (Khung mẫu cố định):** Để phát hiện nhiều vật thể nằm đè lên nhau hoặc có tỉ lệ hình dạng khác nhau (cao/rộng/vuông) tại cùng một vị trí, mô hình định nghĩa trước danh sách các **Anchor Boxes** với kích thước và tỉ lệ khung hình cố định. Mô hình sẽ dự đoán độ chệch (offsets) Δx, Δy, Δw, Δh so với Anchor Box thay vì đoán tự do.

## 2. Các Chỉ số Đánh giá & Thuật toán Cốt lõi

**a) Intersection over Union (IoU):** Chỉ số đo độ trùng khớp giữa Bounding Box dự đoán (B_pred) và Bounding Box thực tế (B_gt):
**IoU = Area(B_pred ∩ B_gt) / Area(B_pred ∪ B_gt)**
Ngưỡng chuẩn: Nếu IoU ≥ 0.5 (hoặc 0.75), khung được tính là **True Positive (TP)**.

**b) Non-Maximum Suppression (NMS):** Thuật toán loại bỏ các Bounding Box dư thừa trùng lặp trên cùng một vật thể:
1. Lọc bỏ tất cả khung có điểm tin cậy (Confidence Score) thấp hơn ngưỡng threshold.
2. Chọn khung có Confidence Score cao nhất (B_max) cho vào danh sách kết quả.
3. Tính IoU giữa B_max và tất cả các khung còn lại của cùng lớp.
4. Loại bỏ các khung có IoU > IoU_threshold (ví dụ 0.5). Lặp lại quy trình.




Hình 1: Minh họa Nguyên lý tính toán Intersection over Union (IoU) giữa Ground Truth và Prediction Box.

## 3. So sánh Hai Họ Mô hình: Two-Stage vs One-Stage Detectors

**1. Họ Two-Stage Detectors (Ưu tiên Độ chính xác):**

• **R-CNN:** Dùng Selective Search tạo 2.000 vùng đề xuất (Region Proposals) -> Đưa từng vùng qua CNN trích xuất đặc trưng -> Phân loại bằng SVM. Rất chậm (~47 giây/ảnh).

• **Fast R-CNN:** Đưa toàn bộ ảnh qua CNN một lần duy nhất -> Dùng lớp **RoI Pooling** để cắt vùng đặc trưng tương ứng từ Feature Map. Tăng tốc ~147x.

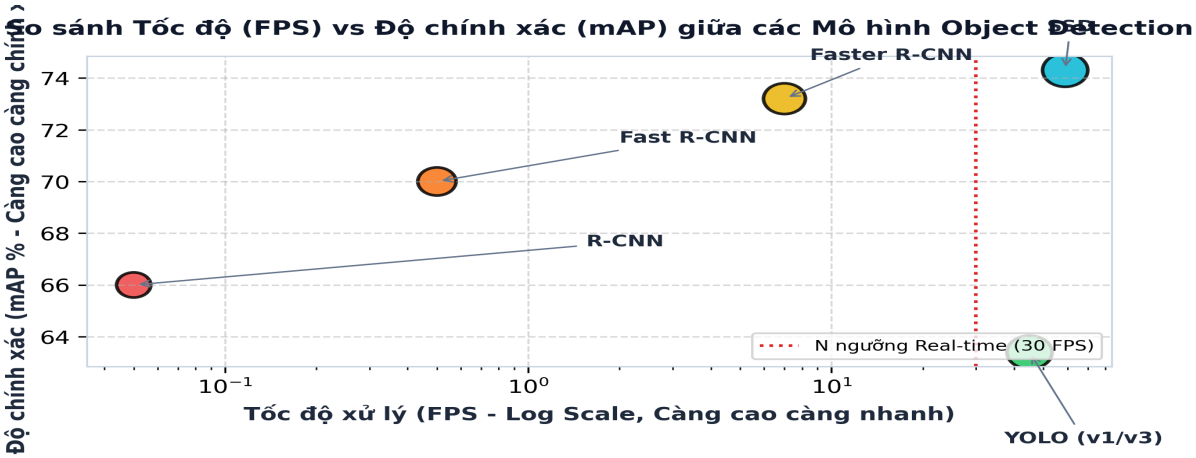
• **Faster R-CNN:** Thay Selective Search bằng mạng **RPN (Region Proposal Network)** kết hợp Anchor Boxes. Mô hình học end-to-end hoàn chỉnh đạt tốc độ ~7 FPS.

**2. Họ One-Stage Detectors (Ưu tiên Tốc độ Real-time):**

• **YOLO (You Only Look Once):** Chia ảnh thành lưới S x S. Mỗi ô lưới dự đoán B Bounding Box và xác suất lớp trực tiếp trong **1 lần lan truyền tiến (Single Forward Pass)**. Tốc độ siêu nhanh (45–155 FPS).

• **SSD (Single Shot MultiBox Detector):** Dự đoán trên nhiều bản đồ đặc trưng ở các tỷ lệ khác nhau (Multi-scale Feature Maps), giúp phát hiện vật thể nhỏ tốt hơn YOLOv1.

| Thuật toán | Kiến trúc | Tốc độ (FPS) | mAP (%) | Đặc điểm nổi bật |
|---|---|---|---|---|
| R-CNN | Two-Stage | 0.05 FPS | 66.0% | Selective Search 2000 vùng, chạy CNN độc lập. |
| Fast R-CNN | Two-Stage | 0.5 FPS | 70.0% | RoI Pooling cắt đặc trưng trên Feature Map chung. |
| Faster R-CNN | Two-Stage | 7.0 FPS | 73.2% | Tích hợp RPN & Anchor Boxes, huấn luyện end-to-end. |
| YOLO (v1/v3) | One-Stage | 45 - 155 FPS | 63.4% | Single Forward Pass, xử lý Video Real-time. |
| SSD | One-Stage | 59.0 FPS | 74.3% | Multi-scale Feature Maps, phát hiện vật thể nhỏ tốt. |




Hình 2: Biểu đồ so sánh Tốc độ xử lý (FPS) vs Độ chính xác (mAP) giữa các thuật toán Object Detection.

## 4. Code Example Python Thực tế (IoU, NMS & PyTorch Inference)

### a) Code Python tính toán IoU (Intersection over Union) chuẩn bằng PyTorch/NumPy:

```
import torch

def compute_iou(box1, box2):
    """
    Tính IoU giữa box1 và box2. Format: [x1, y1, x2, y2]
    """
    # Tọa độ vùng giao (Intersection)
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    # Diện tích vùng giao
    intersection = max(0, x2 - x1) * max(0, y2 - y1)

    # Diện tích từng box
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])

    # Diện tích vùng hợp (Union)
    union = area1 + area2 - intersection

    return intersection / union if union > 0 else 0.0

# Ki□m th□ với ví dụ thực t□:
b1 = [50, 50, 200, 200]
b2 = [100, 100, 250, 250]
print(f"IoU Score: {compute_iou(b1, b2):.4f}") # K□t qu□: ~0.2857
```

### b) Thuật toán Non-Maximum Suppression (NMS) trong Python:

```
import torch

def non_max_suppression(boxes, scores, iou_threshold=0.5):
    """
    boxes: Tensor shape (N, 4) [x1, y1, x2, y2]
    scores: Tensor shape (N,) đi□m tin cậy
    """
    sorted_indices = torch.argsort(scores, descending=True)
    keep_boxes = []

    while len(sorted_indices) > 0:
        current = sorted_indices[0]
        keep_boxes.append(current.item())

        if len(sorted_indices) == 1:
            break

        current_box = boxes[current]
        other_boxes = boxes[sorted_indices[1:]]

        xx1 = torch.maximum(current_box[0], other_boxes[:, 0])
        yy1 = torch.maximum(current_box[1], other_boxes[:, 1])
        xx2 = torch.minimum(current_box[2], other_boxes[:, 2])
        yy2 = torch.minimum(current_box[3], other_boxes[:, 3])

        inter_area = torch.clamp(xx2 - xx1, min=0) * torch.clamp(yy2 - yy1, min=0)
        box1_area = (current_box[2] - current_box[0]) * (current_box[3] - current_box[1])
        box2_area = (other_boxes[:, 2] - other_boxes[:, 0]) * (other_boxes[:, 3] - other_boxes[:, 1])

        iou = inter_area / (box1_area + box2_area - inter_area)
        sorted_indices = sorted_indices[1:][iou <= iou_threshold]

    return keep_boxes
```

### c) Code Inference YOLOv5 với PyTorch Hub (Chỉ với vài dòng mã):

```
import torch
import cv2

# 1. Load mô hình YOLOv5s pre-trained từ PyTorch Hub
model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)

# 2. Thực hiện Object Detection (Inference)
img_path = 'sample_image.jpg'
results = model(img_path)

# 3. Chuy□n k□t qu□ Bounding Box & Class v□ DataFrame
predictions = results.pandas().xyxy[0]
print(predictions[['xmin', 'ymin', 'xmax', 'ymax', 'confidence', 'class', 'name']])

# 4. Vẽ Bounding Box và lưu □nh k□t qu□
results.render()
cv2.imwrite('output_detected.jpg', results.ims[0])
```

## 5. Hướng dẫn Lựa chọn Mô hình theo Yêu cầu Bài toán

**Quy tắc thực tế khi triển khai Object Detection:**

• **Nếu bài toán yêu cầu Real-time (Video, Camera giám sát, Xe tự hành):** Ưu tiên chọn **YOLO (v5/v8)** hoặc **SSD** vì tốc độ xử lý vượt trội (>30 FPS).

• **Nếu bài toán yêu cầu Độ chính xác cao (Chẩn đoán Y tế, Phân tích ảnh Vệ tinh):** Ưu tiên chọn **Faster**

**R-CNN** hoặc **Cascade R-CNN**.
• **Nếu thiết bị phần cứng hạn chế (Edge Devices, Mobile, Raspberry Pi):** Chọn **YOLO-Nano / YOLO-Tiny** hoặc **MobileNet-SSD**.